

Lab 14. Cohere and LangChain – Create a Chatbot

Build a RAG-Powered Chatbot for PDF Documents

Duration	45-60 minutes (core) + 30-40 min optional TO DO extensions
Difficulty	Intermediate
Tech Stack	Node.js (TypeScript), React, LangChain, Cohere API, Chroma Vector DB (container)
Repository	https://github.com/Sangwan70/cohere-chatbot-pdf-chroma
Prerequisites	Node.js ≥ 20, Podman Cohere API key, GitHub account

Objective

Build a Retrieval-Augmented Generation (RAG) chatbot that answers questions grounded in a set of PDF documents. The stack uses Chroma as the vector database (running as a container — no cloud account required), LangChain for orchestration, Cohere for embeddings and generation, and a TypeScript/React frontend.

 *Disclaimer: This project is for study purposes. Some edge-case behaviours (e.g. reference counts) may vary from what you expect. The goal is to understand RAG architecture hands-on.*

Architecture Overview

When a user asks a question, the following pipeline runs:

- PDFs are split into overlapping chunks and converted to embeddings using Cohere Embed.
- Embeddings are stored in a local Chroma vector database running inside a container.
- At query time, the user's question is embedded and matched against stored chunks.
- Relevant chunks are retrieved and passed as context to Cohere Command (the LLM).
- The LLM generates a grounded answer — it only responds if context is available.

Part 1 — Setup & Installation

1. Create an account on Github, if not already existing and login.
2. Visit the repository page

<https://github.com/Sangwan70/cohere-chatbot-pdf-chroma.git>

3. Click on **Fork** button to fork the repo to your Github account, so that you can change the code, upload your PDFs and add/finetune features.
4. Inside **docs** folder (**in your Github account**), upload your pdf files or folders that contain pdf files. You will be ingesting these files as your knowledge base. Commit the changes. **Ensure that the PDF files are not password protected and file names are not space separated.**
5. Open terminal from **Launcher Tab** of your Jupyter Notebook Session.
6. Switch to root user

```
$ sudo su -
```

-
7. **Verify** if `nodejs ^20.19.0 || ^22.13.0 || >=24` is installed. **If not present**, install. Proceed with following steps.

```
node -v
```

```
# dnf erase nodejs -y
```

- a. Download the setup script:

```
curl -fsSL https://rpm.nodesource.com/setup_24.x -o nodesource_setup.sh
```

- b. Run the setup script as root:

```
bash nodesource_setup.sh
```

- c. Install Node.js:

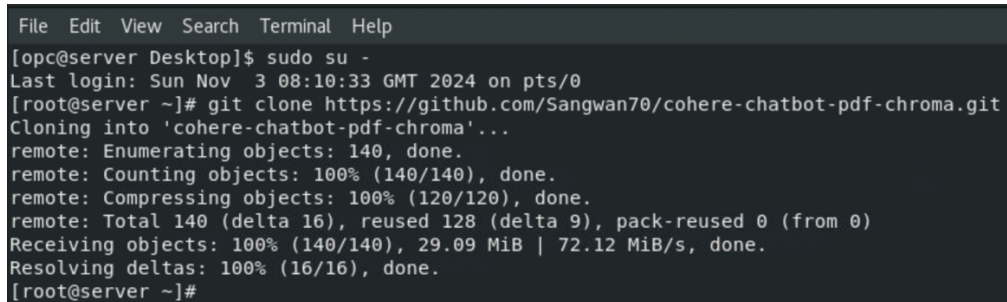
```
dnf install -y nodejs
```

- d. Verify the installation:

```
node -v
```

8. Now clone the forked repo or download the ZIP to your OCI Instance

```
git clone https://github.com/<Your User ID>/cohere-chatbot-pdf-chroma.git
```

A terminal window screenshot showing the cloning of a repository. The terminal has a menu bar with 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The prompt is '[opc@server Desktop]\$' and the user enters 'sudo su -'. The prompt changes to '[root@server ~]#'. The user enters 'git clone https://github.com/Sangwan70/cohere-chatbot-pdf-chroma.git'. The terminal shows the progress of cloning: 'Cloning into \'cohere-chatbot-pdf-chroma\'...', 'remote: Enumerating objects: 140, done.', 'remote: Counting objects: 100% (140/140), done.', 'remote: Compressing objects: 100% (120/120), done.', 'remote: Total 140 (delta 16), reused 128 (delta 9), pack-reused 0 (from 0)', 'Receiving objects: 100% (140/140), 29.09 MiB | 72.12 MiB/s, done.', 'Resolving deltas: 100% (16/16), done.', and the final prompt is '[root@server ~]#'.

9. Change to cloned repository and Install packages

```
cd cohere-chatbot-pdf-chroma
```

10. Install `yarn` globally with:

```
npm install yarn -g
```

11. Run the following command to install all packages from `package.json` file:

```
yarn install && yarn upgrade
```

After installation, you should see a `node_modules` folder.

Part 2 — Configuration

-
1. Get your free trial API Key from <https://dashboard.cohere.com/api-keys> and add to the .env file with following commands:

```
$ vi .env
COHERE_API_KEY=<paste-your-key-here>
COLLECTION_NAME=skillpedia
```

2. The preamble controls your chatbot's personality and scope restrictions. Open:

```
vi utils/makechain.ts
```

Find the QA_TEMPLATE constant (around line 5–19). Edit it to match your use case. Example — a Responsible AI policy assistant:

```
You are a helpful assistant for Oracle's Responsible AI policies. Answer
questions ONLY based on the documents provided. If the answer is not in the
context, say: "I don't have that information in the provided documents."
```

Save and close the file. The system prompt is the single most powerful lever you have for chatbot quality.

Part 3 — Run Chroma and Ingest PDFs

1. Run the Chroma vector database as a container (Podman is pre-installed):

```
podman run -d -p 8000:8000 chromadb/chroma
```

When prompted, select docker.io using the down-arrow key and press Enter.

2. Verify Chroma is running:

```
podman ps
```

3. From the cohere-chatbot-pdf-chroma directory, run:

```
yarn run ingest
```

This script:

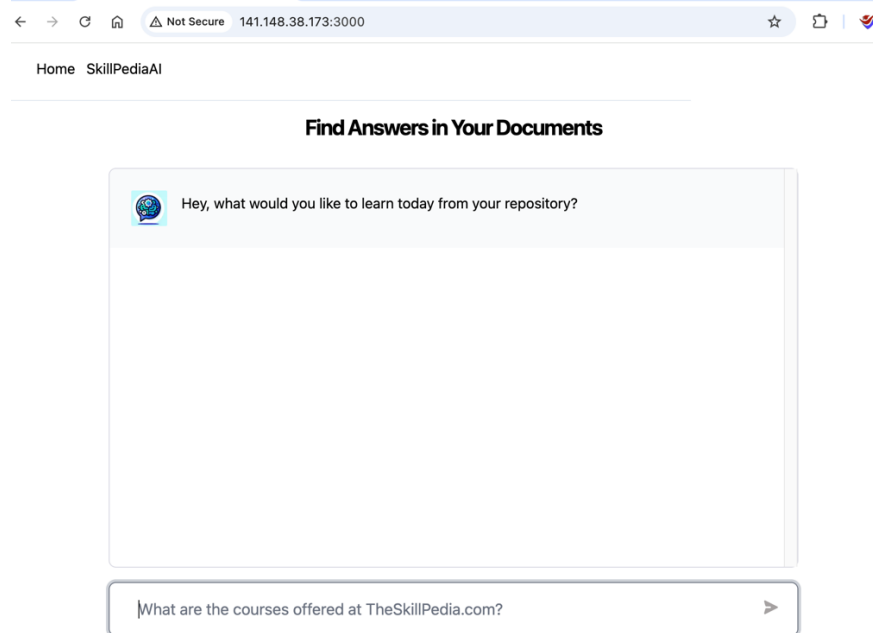
- a. reads all PDFs from the docs/ folder,
- b. splits them into chunks,
- c. embeds them using Cohere, and
- d. stores vectors in Chroma.

Part 4 — Launch and Test the Chatbot

```
npm run dev
```

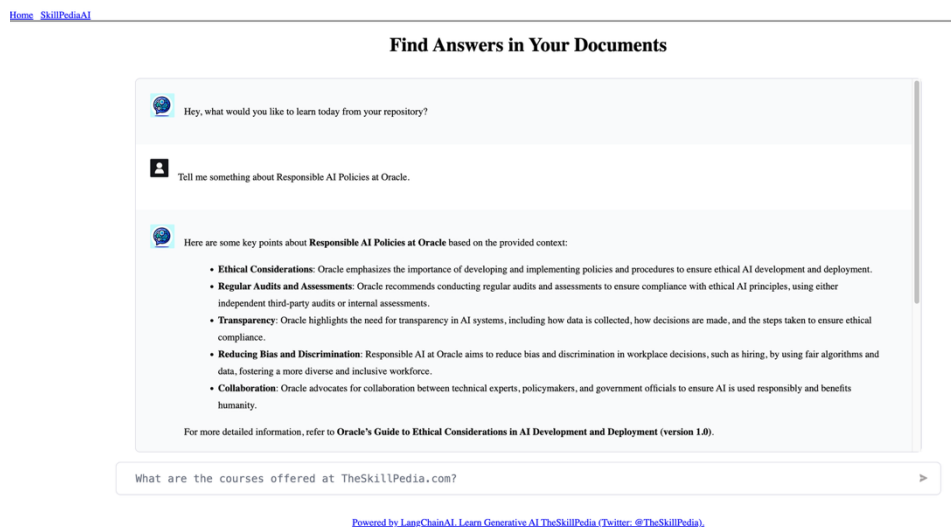
1. Open your browser and use your instance's public IP:

```
http://<Public-IP>:3000
```



2. Ask a Question. Type a question about Responsible AI at Oracle. For example

"Tell me something about Responsible AI Policies at Oracle."



3. Try a question not available in PDF Documents:

"Tell me something about Ram N Sangwan."

The Chatbot will politely refused to answer since we have configured the system prompt to respond only if the question is related to the context provided.

Step 9: Test with Sample Questions

Try these test scenarios:

Query	Expected Behaviour
Tell me about Responsible AI Policies at Oracle.	Answers from PDF context with relevant details.
What are the key principles covered?	Lists principles found in the document.
Tell me something about Ram N Sangwan.	Politely declines — not in context.
What is the capital of France?	Politely declines — not in context.

TO DO — Extend Your Chatbot (30–45 min each)

You've set up the chatbot — now make it yours. The tasks below are designed to deepen understanding and build portfolio-worthy improvements. Each is scoped for 30–45 minutes of focused work.

□ TO DO 1: Add PDF Upload Through the UI

Goal: Let users upload their own PDFs through the browser — no server restarts required.

Currently, PDFs must be placed in the docs/ folder manually before ingestion. Modify the app so users can upload files through the React frontend.

Hints:

- Add a file input (`<input type="file" accept=".pdf">`) to the React frontend.
- Create a new API route `/api/upload` in the Next.js backend to receive files via `FormData`.
- After upload, trigger `yarn run ingest` (or a subset of it) programmatically.
- Bonus: show a progress bar while ingestion runs.

✓ TO DO 1 Checklist

- ☐ Frontend file input added and wired to an upload handler.
- ☐ Backend API route `/api/upload` receives and saves the PDF.
- ☐ Ingestion is triggered automatically after upload.
- ☐ (Bonus) User sees a loading indicator during ingestion.

□ TO DO 2: Implement Conversation Memory

Goal: Make the chatbot remember earlier messages in the same session.

Right now each question is treated independently. LangChain's `ConversationBufferMemory` (or `ConversationalRetrievalChain`) lets the model reference earlier turns.

Hints:

- In `utils/makechain.ts`, switch from `RetrievalQAChain` to `ConversationalRetrievalChain`.

- Pass `chat_history` as input alongside the question.
- Maintain a history array in the React component and append each Q&A pair.
- Test: ask a follow-up like "Tell me more about the second point."

✅ TO DO 2 Checklist

- ☐ `ConversationalRetrievalChain` is wired in `makechain.ts`.
- ☐ Chat history is passed with each request.
- ☐ Follow-up questions successfully reference earlier context.

❑ TO DO 3: Add Source Citations with Page Numbers

Goal: Every answer should show which document chunk it came from.

Cohere and LangChain return `source_documents` metadata with each response. Surface this in the UI.

Hints:

- Return `sourceDocuments` from the LangChain chain response (they are already there).
- In the API response JSON, include a `sources` array with filename and page metadata.
- Display sources below each answer in the React chat interface (e.g., "Source: doc.pdf, p.3").

✅ TO DO 3 Checklist

- ☐ `source_documents` are extracted and returned in the API response.
- ☐ The UI renders a collapsible 'Sources' section under each answer.
- ☐ At least filename is shown; page number is a bonus if metadata includes it.

❑ TO DO 4: Enable Persistent Chroma Storage

Goal: Make embeddings survive container restarts — no re-ingestion needed.

By default, Chroma stores data in memory and loses it when the container stops. Mount a local volume to persist the database.

Hints:

- Stop the current Chroma container.
- Restart with a volume mount:

```
podman run -d -p 8000:8000 \
-v ./chroma-data:/chroma/chroma chromadb/chroma
```

- Re-run `yarn run ingest` once to populate the persistent store.
- Restart the container — verify the data survives without re-ingesting.

✓ TO DO 4 Checklist

- ☐ Chroma container runs with a volume mount (-v flag).
- ☐ yarn run ingest populates the mounted volume.
- ☐ After container restart, the app answers correctly without re-ingesting.

□ TO DO 5: Evaluate and Document Answer Quality

Goal: Build intuition for RAG failure modes through systematic evaluation.

Design a small evaluation set and score your chatbot honestly.

Question	Expected Answer (short)	Actual Answer (short)	Score 1-5

After scoring, answer these reflection questions in 1-2 sentences each:

- What type of questions scored lowest? (Too vague, out of scope, ambiguous?)
- How would changing chunk size or overlap affect these results?
- What would be the business impact if this chatbot were deployed to real users?

✓ TO DO 5 Checklist

- ☐ Evaluation table filled in with at least 5 real test questions.
- ☐ Average score calculated and noted.
- ☐ At least one reflection question answered in writing.

Quick Reference

Key Commands

Command	Purpose
<code>podman run -d -p 8000:8000 chromadb/chroma</code>	Start Chroma container
<code>yarn run ingest</code>	Embed PDFs and load into Chroma

Command	Purpose
<code>npm run dev</code>	Start the dev server (http://localhost:3000)
<code>podman ps</code>	List running containers
<code>podman stop <id></code>	Stop the Chroma container
<code>vi utils/makechain.ts</code>	Edit system prompt / preamble
<code>vi .env</code>	Edit API key and collection name

Useful Links

- **Cohere Dashboard (API Keys):** <https://dashboard.cohere.com>
- **Cohere Docs:** <https://docs.cohere.com>
- **LangChain Docs:** <https://js.langchain.com/docs>
- **Chroma Docs:** <https://docs.trychroma.com>
- **Project Repository:** <https://github.com/Sangwan70/cohere-chatbot-pdf-chroma>
- **lablab.ai — Award-Winning AI Projects:** <https://lablab.ai/apps/recent-winners>

Well done — you've built a production-pattern RAG chatbot!